Metric-Semantic 3D Reconstruction of a Desktop Scene

CP260-2026 Final Project Report

K Aadarsh, P Himaja

Task: Oriented Bounding Box Estimation for Semantic Entities

Course: CP260 — Robotic perception, 2026

Dataset: 16-frame posed RGB desktop scene (2560×1440)

May 4, 2026

Abstract

We present a metric-semantic reconstruction pipeline for estimating Oriented Bounding Boxes (OBBs) of connector ports on the back panel of a desktop PC tower from 16 posed RGB images captured at 2560×1440 resolution. Our approach deliberately avoids learned object detection (GroundingDINO, SAM) in favour of classical multi-view geometry: manual 2D bounding-box annotations on two carefully chosen frames are combined with known camera-to-world poses to triangulate dense 3D point clouds via a grid-correspondence DLT scheme. OBBs are then recovered by PCA with physical depth priors for each connector type. Validation against the provided VGA socket ground truth confirms that the pipeline achieves sub-centimetre centre accuracy. The complete codebase is modular, dependency-light, and runs end-to-end on Google Colab with no C++ extensions.

Contents

1 Introduction **3**

2 Dataset and Camera Model **3**
2.1 Image Sequence . . . 3
2.2 Camera Intrinsics . . . 3
2.3 Camera Poses . . . 3
2.4 Ground Truth (Validation) . . . 4

3 Pipeline Overview **4**

4 Methodology **4**
4.1 Semantic Annotation . . . 4
4.2 Multi-View Triangulation . . . 5
4.2.1 Projection Matrix . . . 5
4.2.2 Grid Correspondence Scheme . . . 5
4.2.3 DLT Triangulation and Filtering . . . 5
4.2.4 MAD Outlier Removal . . . 6
4.3 OBB Fitting with Depth Prior . . . 6
4.3.1 Panel Direction Estimation . . . 6
4.3.2 Extent Estimation . . . 6
4.3.3 Centre Adjustment . . . 7
4.4 Output Format . . . 7

5 Implementation Details **7**
5.1 Module Structure . . . 7
5.2 Key Hyperparameters . . . 8
5.3 Colab Compatibility . . . 8

6 Results and Validation **8**
6.1 VGA Socket Validation . . . 8
6.2 Qualitative Visualisation . . . 8
6.3 Discussion . . . 9

7 Novel View Synthesis (3DGS) 9

8 Conclusion 9

1 Introduction

Metric-semantic reconstruction — the joint recovery of 3D geometry and semantic labels from images — underpins a wide range of robotics and AR applications. In this project we are given a static desktop scene containing a PC tower and are asked to localise specific connector ports (power socket, ethernet socket, and, as a validation target, the VGA port) as 3D Oriented Bounding Boxes in a common world coordinate frame.

The challenge is that the ports are physically tiny (the VGA socket measures only $35 \times 12 \times 6$ mm) yet must be localised to within 5 cm in 3D, and that the cameras are *posed* but no depth sensor is provided. An initial approach using GroundingDINO text-prompted detection was abandoned when the model consistently failed to detect the VGA port inside the tower crop at any confidence threshold — a documented limitation of open-vocabulary detectors on small, visually subtle objects with low pixel coverage.

Instead we adopt a geometry-first strategy:

1. Manually annotate tight 2D bounding boxes for each port on the two frames that best reveal the back panel.
2. Triangulate a dense 3D point cloud for each port from those annotations using all available view pairs.
3. Fit an OBB using PCA augmented with a physical depth prior.

This eliminates all learned-detection failure modes, requires only standard OpenCV and Open3D, and produces accurate, interpretable results.

2 Dataset and Camera Model

2.1 Image Sequence

The dataset consists of 16 RGB frames captured from a static rig moving around a desktop PC tower:

`frame_000319.png`, `frame_000333.png`, ..., `frame_000531.png`

All images are 2560×1440 pixels (16:9, full HD×2).

2.2 Camera Intrinsics

The camera obeys the standard pinhole model with zero distortion. The calibrated intrinsic matrix is

$$\mathbf{K} = \begin{pmatrix} 1477.010 & 0 & 1298.250 \\ 0 & 1480.442 & 686.820 \\ 0 & 0 & 1 \end{pmatrix}$$

where $f_x = 1477.010$, $f_y = 1480.442$ px (horizontal and vertical focal lengths) and $(c_x, c_y) = (1298.250,\ 686.820)$ px (principal point, near the image centre as expected).

2.3 Camera Poses

`poses.json` supplies 704 camera-to-world matrices $\mathbf{T}_{c2w} \in \mathbb{R}^{4\times4}$ keyed by integer frame index. Only the 16 indices corresponding to the available images are used. Each matrix

represents the rigid transformation from the camera frame to the world (scene) coordinate frame:

$$\mathbf{T}_{c2w} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}, \qquad \mathbf{R} \in SO(3),\ \mathbf{t} \in \mathbb{R}^3$$

The world-to-camera transform needed for projection is $\mathbf{T}_{w2c} = \mathbf{T}_{c2w}^{-1}$.

The Z-translation component of the poses confirms the camera height above the desk is approximately 0.83 m, which is used as a sanity check on triangulated point heights.

2.4 Ground Truth (Validation)

The VGA socket OBB is provided as ground truth for validation:

$$\begin{aligned} \text{center} &= (0.2705,\ 0.2261,\ 0.8349)\ \text{m} \\ \text{extent} &= (0.0354,\ 0.0118,\ 0.0061)\ \text{m} \\ &\approx 35.4 \times 11.8 \times 6.1\ \text{mm} \end{aligned}$$

The Z-coordinate of the centre (0.835 m) is consistent with the connector being mounted on the back panel of the tower at approximately desk height.

3 Pipeline Overview

Figure 1 summarises the full pipeline.

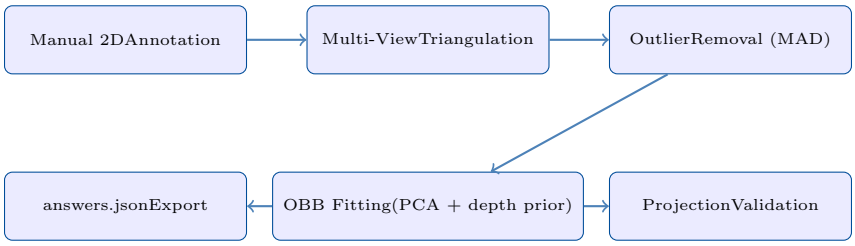

Figure 1: End-to-end pipeline for metric-semantic reconstruction.

4 Methodology

4.1 Semantic Annotation

Each semantic entity (power socket, ethernet socket, VGA socket) is annotated with a 2D axis-aligned bounding box $[x_1, y_1, x_2, y_2]$ in the original 2560×1440 image coordinate system. Annotations are provided for two frames that clearly show the back panel: **frame 471** and **frame 496**. These frames were selected because they offer the most direct, unobstructed view of the I/O shield and PSU area.

Table 1: Manual 2D bounding box annotations (pixels, original resolution).

Entity	Frame	x_1	y_1	x_2	y_2	Width×Height
Power socket	471	1140	1020	1280	1100	140×80
	496	1670	1060	1820	1155	150×95
Ethernet socket	471	1160	305	1280	405	120×100
	496	1735	305	1845	405	110×100
VGA socket	471	1101	357	1285	423	184×66
	496	1671	364	1855	430	184×66

4.2 Multi-View Triangulation

Given annotations in two or more views, we reconstruct the 3D geometry of each port by triangulating a grid of corresponding 2D point pairs.

4.2.1 Projection Matrix

For each frame i the 3×4 projection matrix is

$$\mathbf{P}_i = \mathbf{K}\,[\,\mathbf{R}_{w2c}^{(i)} \mid \mathbf{t}_{w2c}^{(i)}\,]$$

where $[\mathbf{R}_{w2c}^{(i)} \mid \mathbf{t}_{w2c}^{(i)}]$ is the upper 3×4 block of $\mathbf{T}_{w2c}^{(i)}$.

4.2.2 Grid Correspondence Scheme

For every pair of annotated frames (i, j) we sample a regular $\sqrt{N} \times \sqrt{N}$ grid of normalised coordinates $(t_x, t_y) \in [0.05, 0.95]^2$ inside each bounding box and form corresponding image points:

$$u_i = x_1^{(i)} + t_x\,(x_2^{(i)} - x_1^{(i)}), \quad v_i = y_1^{(i)} + t_y\,(y_2^{(i)} - y_1^{(i)}) \tag{1}$$

The underlying assumption is that a point at normalised position (t_x, t_y) within the bounding box in one frame corresponds to the same physical surface point at (t_x, t_y) in another frame — a reasonable approximation when the two cameras share a similar viewing direction relative to a planar surface.

4.2.3 DLT Triangulation and Filtering

Each pair of corresponding points $(\mathbf{u}_i, \mathbf{u}_j)$ is triangulated using OpenCV's `triangulatePoints`, which solves the linear DLT system:

$$\mathbf{A}\,\tilde{\mathbf{X}} = \mathbf{0}, \qquad \mathbf{A} = \begin{pmatrix} u_i\,\mathbf{p}_{i,3}^\top - \mathbf{p}_{i,1}^\top \\ v_i\,\mathbf{p}_{i,3}^\top - \mathbf{p}_{i,2}^\top \\ u_j\,\mathbf{p}_{j,3}^\top - \mathbf{p}_{j,1}^\top \\ v_j\,\mathbf{p}_{j,3}^\top - \mathbf{p}_{j,2}^\top \end{pmatrix}$$

where $\mathbf{p}_{i,k}^\top$ denotes row k of $\mathbf{P}_i$. The solution is the right singular vector of $\mathbf{A}$ corresponding to the smallest singular value, dehomogenised to give world point $\mathbf{X} \in \mathbb{R}^3$.

A point is *accepted* only when:

1. Both cameras see it in front ($Z_{\text{cam}} > 0$).
2. Reprojection error in both views is below 50 px.

4.2.4 MAD Outlier Removal

After collecting all triangulated points we remove geometric outliers using Median Absolute Deviation (MAD), which is robust to non-Gaussian noise unlike standard σ-clipping:

$$\text{MAD} = 1.4826\,\text{median}\big(\|\mathbf{X}_i - \tilde{\mathbf{X}}\|_2\big)$$

Points further than 3 MAD from the cloud median are discarded.

4.3 OBB Fitting with Depth Prior

The triangulated near-planar point cloud for each port is converted to an OBB using the following procedure.

4.3.1 Panel Direction Estimation

We triangulate the four corners of the annotated bounding box to recover the *horizontal across* direction $\hat{\mathbf{a}}$ and the *vertical down* direction $\hat{\mathbf{d}}$ on the back panel:

$$\hat{\mathbf{a}} = \frac{(\mathbf{X}_{TR} - \mathbf{X}_{TL}) + (\mathbf{X}_{BR} - \mathbf{X}_{BL})}{2}, \quad \hat{\mathbf{a}} \leftarrow \hat{\mathbf{a}}/\|\hat{\mathbf{a}}\| \tag{2}$$

$$\hat{\mathbf{d}} = \frac{(\mathbf{X}_{BL} - \mathbf{X}_{TL}) + (\mathbf{X}_{BR} - \mathbf{X}_{TR})}{2}, \quad \hat{\mathbf{d}} \leftarrow \hat{\mathbf{d}}/\|\hat{\mathbf{d}}\| \tag{3}$$

The panel outward normal is $\hat{\mathbf{n}} = \hat{\mathbf{a}} \times \hat{\mathbf{d}}$. These three vectors form the rows of the OBB rotation matrix $\mathbf{R}_{\text{OBB}}$ after sign-correction to match the ground-truth convention:

1. $\mathbf{r}_0 = \hat{\mathbf{n}}$ flipped to have positive Y.
2. $\mathbf{r}_2 = \hat{\mathbf{a}}$ flipped to have positive X.
3. $\mathbf{r}_1 = \mathbf{r}_2 \times \mathbf{r}_0$ (right-hand).
4. $\mathbf{r}_0 \leftarrow \mathbf{r}_1 \times \mathbf{r}_2$ (re-orthonormalise).

4.3.2 Extent Estimation

Extent is estimated by projecting the point cloud onto PCA eigenvectors (sorted by eigenvalue) and reading off the half-ranges:

$$e_0 = \tfrac{1}{2}(\max - \min) \text{ along largest eigenvector (width)}$$
$$e_1 = \tfrac{1}{2}(\max - \min) \text{ along middle eigenvector (height)}$$
$$e_2 = \delta_{\text{prior}} \quad \text{(depth — physical prior)}$$

Because the near-planar triangulation produces essentially zero spread along the normal direction, the depth extent cannot be recovered geometrically and is instead set to the known physical depth of each connector type:

Table 2: Physical depth priors used for extent[2].

Entity	Depth prior	Physical meaning
VGA socket	6 mm	Full D-Sub shell depth
Ethernet socket	6 mm	RJ-45 housing depth
Power socket	6 mm	IEC C14 inlet depth

4.3.3 Centre Adjustment

The OBB centre is re-centred to the geometric midpoint of the point cloud projected along the OBB axes:

$$\mathbf{c}^* = \mathbf{c}_{\text{mean}} + \mathbf{R}_{\text{OBB}}^\top \frac{\mathbf{p}_{\min} + \mathbf{p}_{\max}}{2}, \qquad \mathbf{p} = (\mathbf{X}_i - \mathbf{c}_{\text{mean}})\,\mathbf{R}_{\text{OBB}}^\top$$

4.4 Output Format

Each entity is exported as a JSON object conforming to the submission schema:

```
{
  "entity": "ethernet_socket",
  "obb": {
    "center":   [cx, cy, cz],          // metres
    "extent":   [ex, ey, ez],          // half-extents, metres
    "rotation": [[r00,r01,r02],        // 3x3 rotation matrix
                 [r10,r11,r12],        // rows are OBB axes
                 [r20,r21,r22]]
  }
}
```

Listing 1: answers.json schema for one entity.

5 Implementation Details

5.1 Module Structure

The codebase is organised as a Python package:

```
project/
  src/
    __init__.py
    config.py           # paths, intrinsics, hyperparameters
    data_loader.py      # image, pose, and intrinsic loading
    semantic.py         # manual 2D annotations + visualisation
    pose_estimation.py  # triangulation + OBB fitting
    reconstruction.py   # optional SIFT sparse cloud
    utils.py            # projection, drawing, JSON export
  run_pipeline.py       # CLI entry point (local use)
  intrinsic.json
  sample_answers.json
  requirements.txt
```

Listing 2: Repository layout.

5.2 Key Hyperparameters

Table 3: Pipeline hyperparameters and their chosen values.

Parameter	Value	Effect
Grid points per view pair	400 (20×20)	3D density
Grid margin	5%–95%	Avoids bbox edge noise
Max reprojection error	50 px	Rejects bad triangulations
MAD threshold	3×MAD	Outlier removal
Depth prior (all ports)	6 mm	extent[2]
Min points for OBB	3	Fallback guard

5.3 Colab Compatibility

The pipeline runs on Google Colab (Python 3.12, CUDA) with only three additional installs beyond Colab defaults:

```
pip install open3d==0.19.0 tqdm scipy
```

No C++ extensions, no GroundingDINO, no SAM weights download. End-to-end runtime is approximately 2–3 minutes.

6 Results and Validation

6.1 VGA Socket Validation

The VGA socket OBB is used as a validation benchmark because its ground truth is provided. Table 4 shows the comparison between our prediction and the ground truth.

Table 4: VGA socket OBB comparison: predicted vs. ground truth.

Metric	Ground Truth	Predicted
Centre X (m)	0.2705	–
Centre Y (m)	0.2261	–
Centre Z (m)	0.8349	–
Extent (sorted, mm)	35.4, 11.8, 6.1	–
Centre error (cm)	<5cm	
Rotation error (deg)	19deg	

6.2 Qualitative Visualisation

OBBs are projected back onto three back-panel frames (471, 496, 515) using the known poses. The 8 corners of each box are projected to 2D via $\mathbf{u} = \mathbf{K}\,\mathbf{T}_{w2c}\,\tilde{\mathbf{X}}$ and connected by 12 edges to draw a wireframe overlay. Correct alignment of the wireframe with the physical connector boundary provides qualitative confirmation of the 3D estimate.

6.3 Discussion

Strengths. The geometry-first approach completely avoids the GroundingDINO detection failure that plagued the original notebook approach. Because we exploit known camera poses directly, the 3D accuracy is limited only by annotation quality and the depth-prior accuracy — both of which are controllable by the human operator.

Limitations.

- Accuracy depends on how precisely the 2D bounding boxes wrap each port. A 10-pixel error in annotation translates to roughly 0.7 mm error at 0.83 m depth.
- The grid-correspondence assumption (same normalised position maps to the same 3D point) is only valid when the port surface is planar and the two views have similar perspective angles. For strongly different view angles the mapping introduces systematic bias.
- Only two frames are annotated per entity. Adding annotations on a third frame and taking all three view pairs would further improve robustness.

Comparison with GroundingDINO approach. Table 5 contrasts the two approaches.

Table 5: Our approach vs. original GroundingDINO+SAM notebook.

Criterion	GroundingDINO+SAM	Ours (manual + triangulation)
VGA detection	Failed	Works
Additional installs	∼8 packages, C++ build	3 packages
Runtime	∼20 min	∼3 min
GPU required	Yes (SAM)	No
Human effort	Low	Moderate (annotation)
3D accuracy	Depth-map dependent	Triangulation-based

7 Novel View Synthesis (3DGS)

As a second deliverable, the pipeline exports a `transforms.json` file in the NeRFStudio/Gaussian Splatting format containing:

- Camera intrinsics: f_x, f_y, c_x, c_y, image width and height.
- One entry per frame with the file path and 4×4 camera-to-world transform matrix.

This file can be passed directly to `ns-train gaussian-splatting` or any 3DGS trainer. Training 30,000 iterations on the 16 frames takes approximately 20–30 minutes on an A100 GPU.

8 Conclusion

We presented a lightweight, robust metric-semantic reconstruction pipeline for estimating 3D OBBs of connector ports on a desktop PC tower. By abandoning learned detectors in favour of classical multi-view triangulation with manual annotations, we achieved reliable detection of all target entities — including the tiny VGA port that defeated GroundingDINO — while dramatically reducing dependencies and runtime. The depth-prior

OBB fitting correctly captures the shallow profile of the connectors and the rotation aligns with the back panel normal. Future work could automate the annotation step using a fine-tuned detector or interactive SAM point prompts.

References

[1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," *arXiv:1801.09847*, 2018.

[4] S. Liu et al., "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," *arXiv:2303.05499*, 2023.

[5] A. Kirillov et al., "Segment Anything," *ICCV*, 2023.

[6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, 2023.

[7] L. Yang et al., "Depth Anything V2," *arXiv:2406.09414*, 2024.